Conditional_TF

November 15, 2022

0.0.1 Simulation Investigation into Conditional Trend Filtering and Changepoint Detection on High-Frequency Price Process

Jason Bohne

Jason.bohne@stonybrook.edu

Applied Math and Stats Ph.D. Student

Under Supervision of Professor Pawel Polak

[1]:
```
import sys
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt


path='/home/jbohn/jupyter/personal/'
sys.path.append(f'{path}TAQ-Query-Scripts/data')
sys.path.append(f'{path}Changepoint_LOB/Lasso/CPD')

from kernel_smoother import smooth_series, cv_block_size, mom_kde
from adaptive_tf_ir import l1tf_adaptive_ir, Dmat
```

0.0.2 Prep Data

- First minute seconds of trade data for AAPL after open across availble U.S. equity exchanges
- Group observations by participant side timestamp; averaging price and summing volume
- Results in ~1400 observations
- Preprocessed and cleaned to remove trades executed before open

[2]:
```
trades=pd.read_csv(f'{path}TAQ-Query-Scripts/data/cleaned_trades.
 ↪csv',index_col=0).dropna()
trades=trades[trades.index < "2020-01-02 09:31:00"]
trades=trades[trades.index> "2020-01-02 09:30:00"]
trades.index=pd.to_datetime(trades.index)
trades['Index']=trades.index
trades=trades.groupby('Index').agg({'Trade_Price':np.mean, 'Trade_Volume':np.
 ↪sum})
```

```
trades['Trade_Price']=np.log(trades['Trade_Price'])
trades=trades[trades['Trade_Volume']<1000]


trades
```

[2]:

```
                           Trade_Price  Trade_Volume
Index
2020-01-02 09:30:00.134336     5.691170           250
2020-01-02 09:30:00.134371     5.691170            50
2020-01-02 09:30:00.134532     5.691069           108
2020-01-02 09:30:00.157191     5.691305             2
2020-01-02 09:30:00.193558     5.691136             5
…                                   …           …
2020-01-02 09:30:59.613591     5.689481             5
2020-01-02 09:30:59.613627     5.689469           230
2020-01-02 09:30:59.613659     5.689481            40
2020-01-02 09:30:59.614088     5.689481            30
2020-01-02 09:30:59.614187     5.689447             1

[1349 rows x 2 columns]
```

0.0.3 Step 1; Generate Prior

- In our analysis trade volume will be used as a prior for changes in underlying price trend however any discrete series can be utilized (realized volatility)
- Normalize time of prior to be incremented in total seconds

[3]:

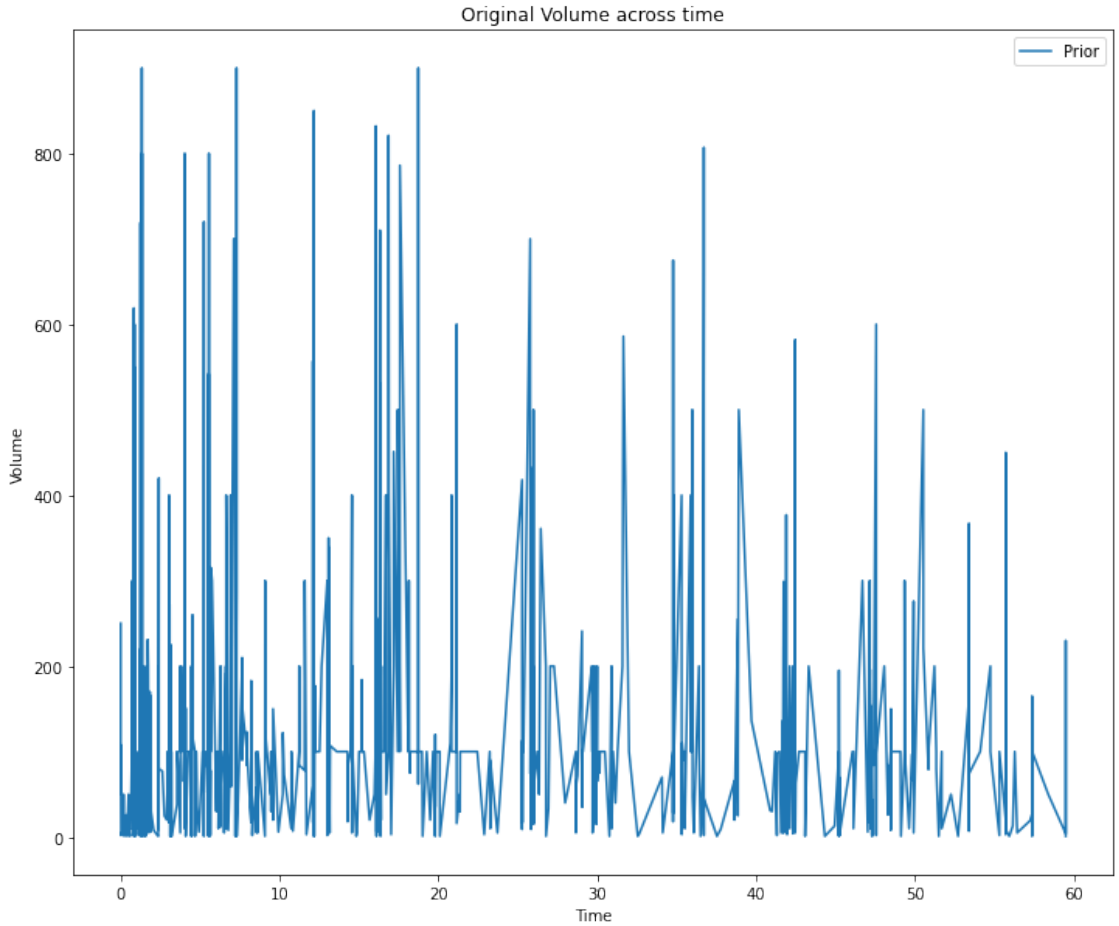
```
original_index=trades['Trade_Volume'].index
volume_index=[(i -original_index[0]).total_seconds() for i in original_index]
volume_prior=trades['Trade_Volume'].values
```

[4]:

```
fig,ax=plt.subplots(figsize=(12,10))
plt.title("Original Volume across time")
plt.ylabel("Volume")
plt.xlabel("Time")
plt.plot(volume_index,volume_prior,label="Prior")
plt.legend()
```

[4]:

```
<matplotlib.legend.Legend at 0x7febb7e9e590>
```



0.0.4 Step 2; Smooth Prior

- Original prior is discrete and experiences high frequency of jumps
- Apply smoothing for better interpretability

Kernel Density Smoothing - Nonparametric smoothing method which smooths candidate point $ x_i$ as a weighted average of all observations x_j where weights w_i are a function of their distance in time from candidate point - Gaussian Kernels specify function of a normal density - Optimal bandwidth is selected either through Scott's or Silvermans.

[5]:
```
kde_smooth,optimal_bw=smooth_series(volume_prior,volume_index)
```

0.0.5 Step 3; Add Robustness to Estimator

- Outliers can significantly skew smoothing estimator
- Apply Median of Means which partitions observations into N blocks
- For each block apply kernel density estimation to generate kernel k_i; estimating candidate point as the median of the $k_j \forall j$
- Cross validate hyperparameter N; specifying number of block partitions

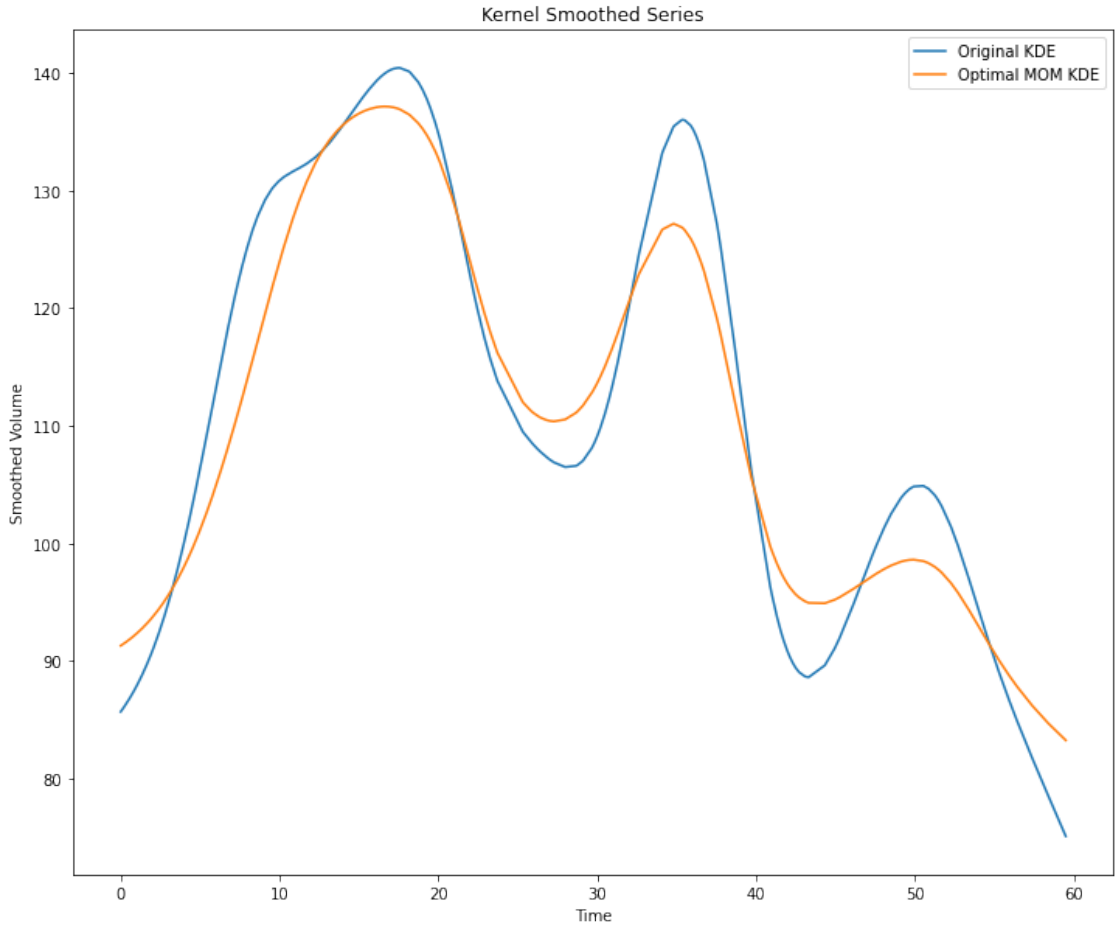
[6]:
```
block_params=np.arange(1,100,10)
optimal_mom_kde_bw,optimal_mse=cv_block_size(volume_prior,kde_smooth,volume_prior,optimal_bw,b
```

```
MSE for 1 blocks is 24838808.36
MSE for 11 blocks is 25222352.25
MSE for 21 blocks is 25254313.25
MSE for 31 blocks is 25258143.59
MSE for 41 blocks is 25259506.4
MSE for 51 blocks is 25261407.14
MSE for 61 blocks is 25261849.4
MSE for 71 blocks is 25267855.22
MSE for 81 blocks is 25270696.1
MSE for 91 blocks is 25270394.88
```

[7]:
```
optimal_mom_kde=mom_kde(kde_smooth,volume_index,optimal_mom_kde_bw,optimal_bw)
```

[8]:
```
fig,ax=plt.subplots(figsize=(12,10))

plt.title("Kernel Smoothed Series")
plt.ylabel("Smoothed Volume")
plt.xlabel("Time")
plt.plot(volume_index,kde_smooth,label="Original KDE")
plt.plot(volume_index,optimal_mom_kde,label="Optimal MOM KDE")
plt.legend()
```

[8]:
```
<matplotlib.legend.Legend at 0x7febc9f8c760>
```



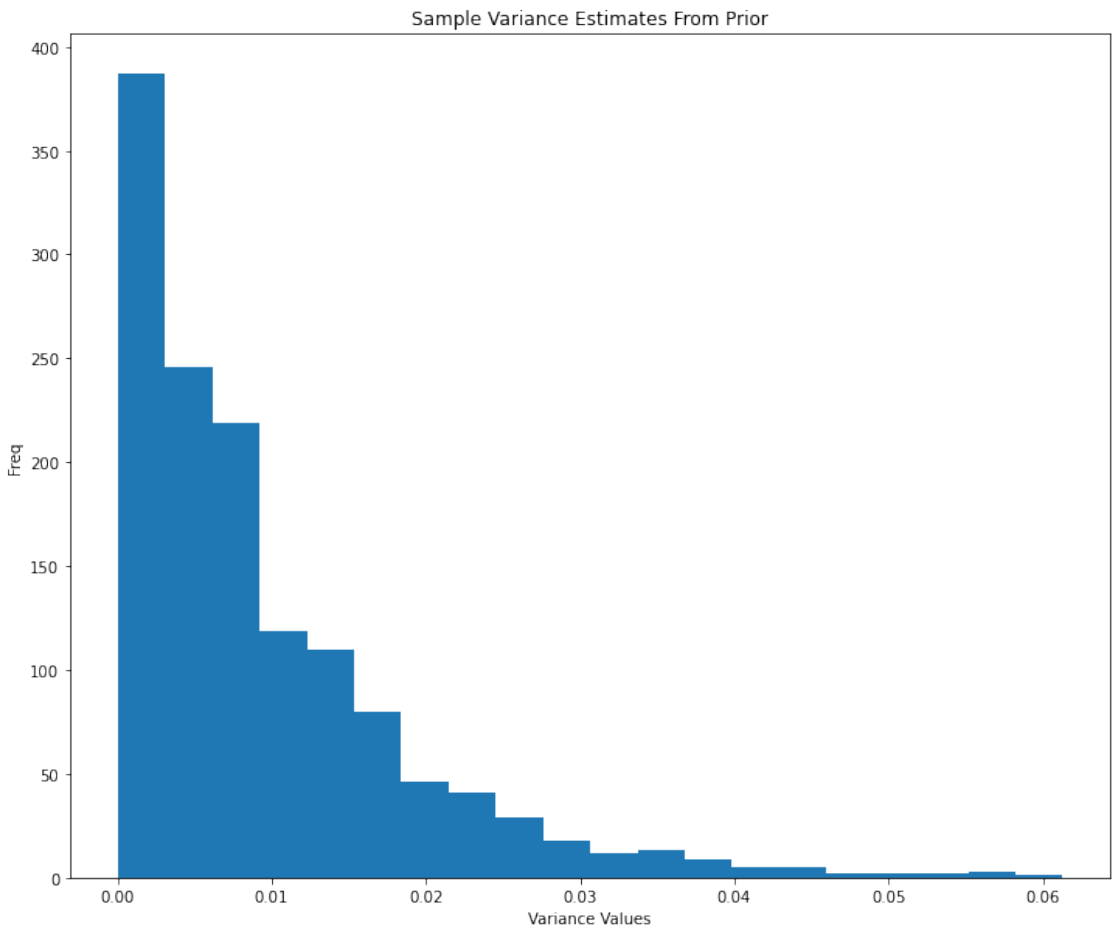
0.0.6 Step 4; Sample Variance process

- As we believe the prior is responsible for changes in the underlying trend; we simulate the variance process of the underlying trend using the prior series
- Larger prior values (trade volume) correspond to larger variance in underlying trend
- Smaller prior values (trade volume) correspond to smaller variance in underlying trend

[9]:
```
def sample_from_prior(smooth_prior):
    """ Sample variance process given smoothed prior
    """
    samples=[]
    for x_i in smooth_prior:
        scale=2/(x_i**2)
        s_i=(x_i/2)*np.random.exponential(scale=scale)

        samples.append(s_i)

    return samples
```

[10]:
```
variance_estimates=sample_from_prior(optimal_mom_kde)
```

[11]:
```
fig,ax=plt.subplots(figsize=(12,10))
plt.title("Sample Variance Estimates From Prior")
plt.xlabel("Variance Values")
plt.ylabel("Freq")
bins=plt.hist(variance_estimates,bins=20)
```



0.0.7 Step 5 ; Simulate underlying trend given variance estimates

- Given variance estimates we can simulate price series across time
- Each value X_t follows a normal distribution of mean $2X_{t-1} - XX_{t-2}$ with variance dictated by estimates above
- Interpretation is that most consecutive values will be piecewise linear with small noise; with sparse jumps arising from larger variance

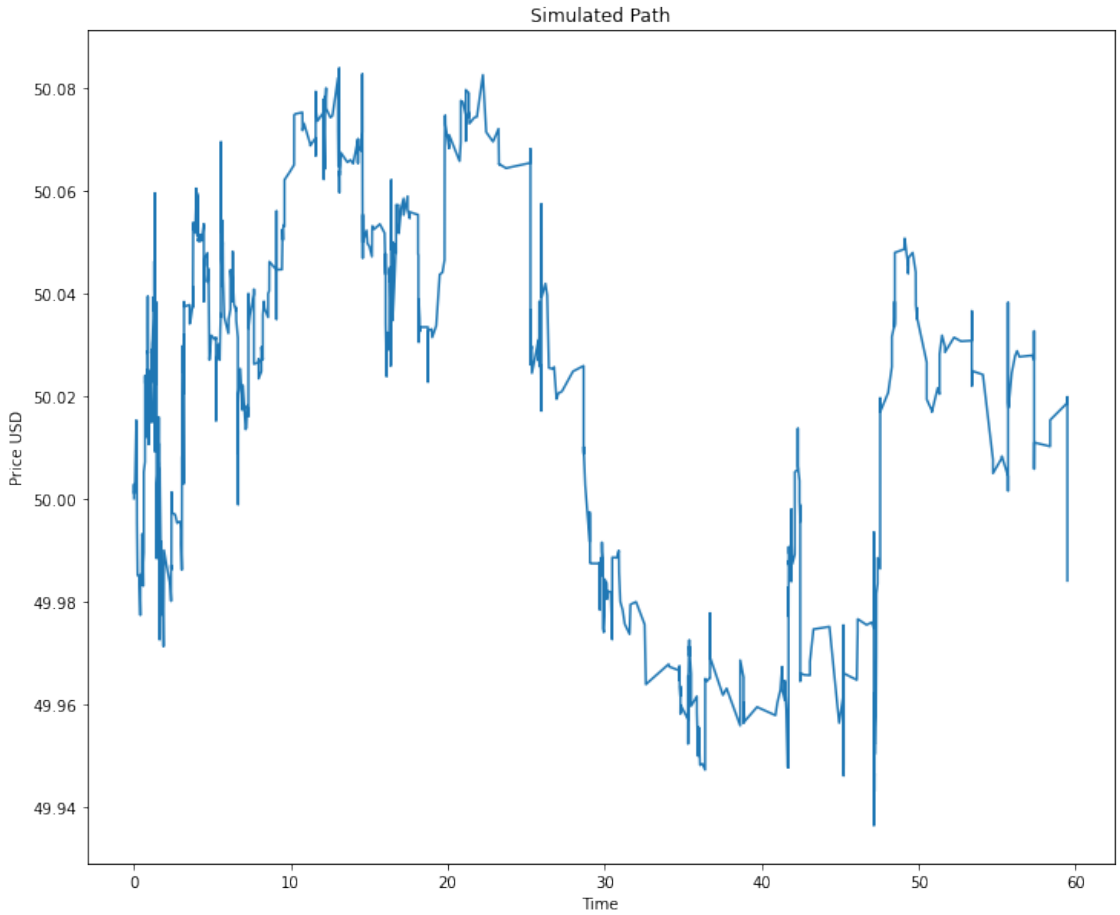
[12]:
```
def sample_price_series(variance_estimates,variance=1,init=0):
```

```
    """ Sample price series by given sparse series representing discrete␣
↪variance process across time
    """

    obs=[]
    obs.append(0)
    obs.append(0)
    for count,param in enumerate(variance_estimates):
        mu=2*obs[count-1]-obs[count-2]

        scale=np.sqrt(param)*variance

        sample=np.random.normal(loc=mu,scale=scale,size=1)[0]

        obs.append(-1*(sample-mu) )

    obs=np.cumsum(np.array(obs))+init

    return obs[1:-1]
```

[13]:
```
price_series=sample_price_series(variance_estimates,variance=0.05,init=50)
```

[14]:
```
fig,ax=plt.subplots(figsize=(12,10))
plt.plot(volume_index,price_series)
plt.xlabel("Time")
plt.ylabel("Price USD")
plt.title("Simulated Path")
```

[14]: Text(0.5, 1.0, 'Simulated Path')



0.0.8 Numerical Result 1; L1 trend filtering

- Given the simulated path from step 5; we wish to apply L1 trend filtering to the problem
- Incorporates irregular spacing between observations
- Constrains our estimator to be piecewise linear
- Here the penalty is set to a constant; dictated by our sensitivity to number of changepoints
  - Hyperparameter can be cross validated

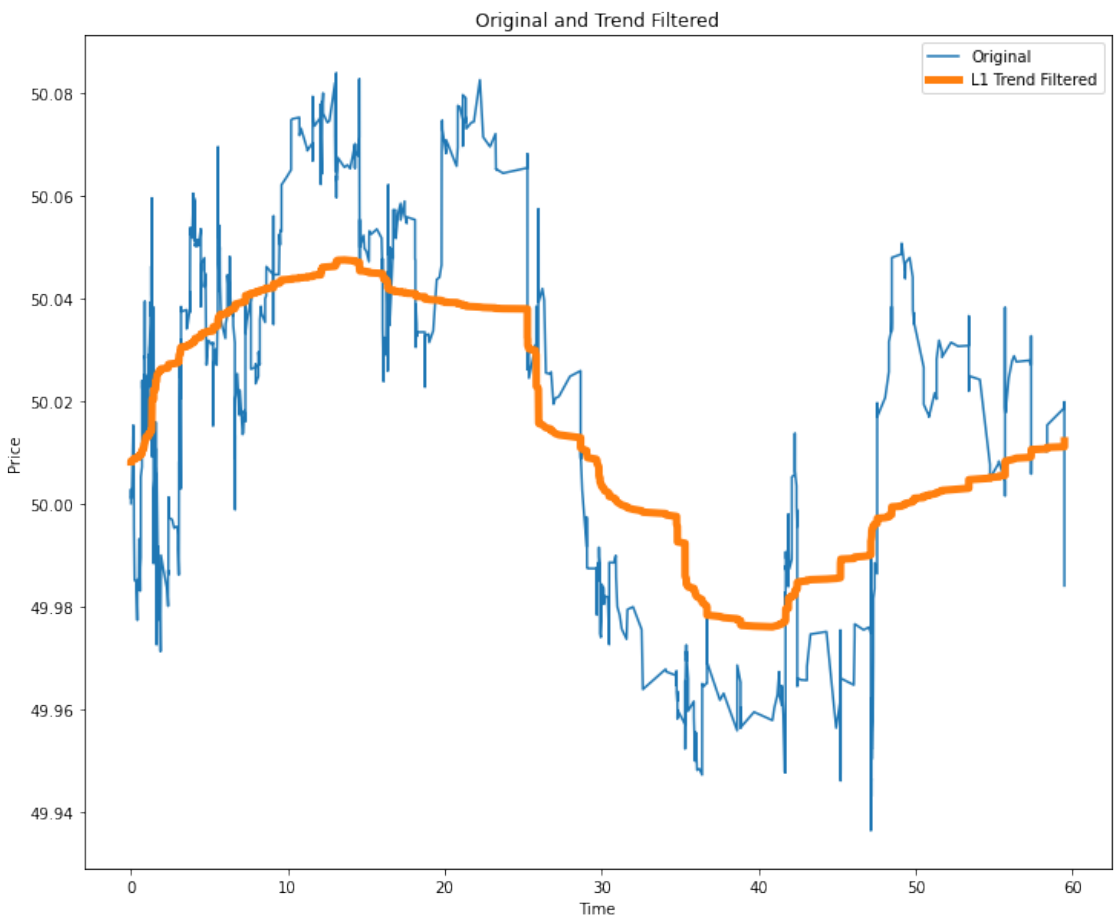
[23]:
```
trend_filter_series,status,D=l1tf_adaptive_ir(price_series.
↪reshape(-1,1),t=volume_index,lambda_p=50)
```

```
pobj1: [[8.35112657e+09]], pobj2: [[20000365.21249915]], dobj: [[0.]], gap:
[[20000365.21249915]]
pobj1: [[1.00977217e+13]], pobj2: [[2.22723166]], dobj: [[0.25133486]], gap:
[[1.9758968]]
pobj1: [[1.68214059e+13]], pobj2: [[0.78958043]], dobj: [[0.39134015]], gap:
[[0.39824027]]
pobj1: [[6.5155647e+12]], pobj2: [[0.71972433]], dobj: [[0.40083659]], gap:
[[0.31888774]]
```

```
pobj1: [[2.87252936e+12]], pobj2: [[0.63994974]], dobj: [[0.42573094]], gap:
[[0.2142188]]
pobj1: [[2.3846599e+14]], pobj2: [[0.60066943]], dobj: [[0.42833497]], gap:
[[0.17233447]]
pobj1: [[3.21151669e+12]], pobj2: [[0.55706852]], dobj: [[0.43274107]], gap:
[[0.12432745]]
pobj1: [[5.0871731e+09]], pobj2: [[0.48790668]], dobj: [[0.47301663]], gap:
[[0.01489005]]
solved
pobj1: [[4.62319675e+09]], pobj2: [[0.48347081]], dobj: [[0.47782936]], gap:
[[0.00564146]]
```

[24]:
```
fig,ax=plt.subplots(figsize=(12,10))
plt.plot(volume_index,price_series,label='Original')
plt.plot(volume_index,trend_filter_series,label='L1 Trend Filtered',lw=5)
plt.title("Original and Trend Filtered")
plt.xlabel("Time")
plt.ylabel('Price')
plt.legend()
```

[24]: <matplotlib.legend.Legend at 0x7febb7ee6290>



0.0.9 Numerical Result 1; Conditional Adaptive L1 trend filtering

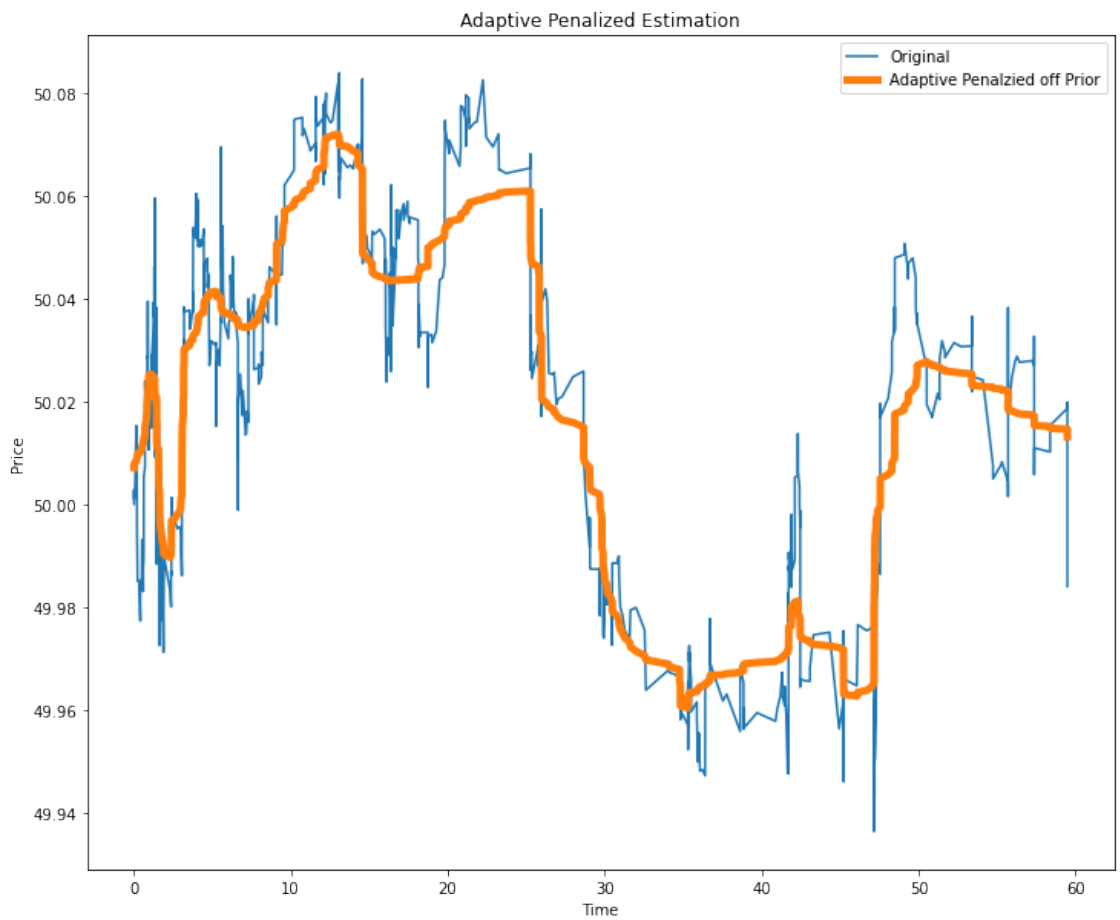
- Estimator is piecewise linear accounting for irregular observations
- Sensitivity of estimator to changes in underlying trend is conditional on inverse of prior
- Included explicitly as adaptive penalty (scaled by scale_factor)

[25]:
```
scale_factor=10e1
adaptive_trend_filter_series,status,D=l1tf_adaptive_ir(price_series.
↪reshape(-1,1),t=volume_index,lambda_p=scale_factor*1/optimal_mom_kde[1:-1])
```

```
pobj1: [[1.16162586e+09]], pobj2: [[398077.40076536]], dobj: [[0.]], gap:
[[398077.40076536]]
pobj1: [[4.20556158e+15]], pobj2: [[0.57241143]], dobj: [[0.09948881]], gap:
[[0.47292262]]
pobj1: [[4.89968839e+15]], pobj2: [[0.32044875]], dobj: [[0.13921131]], gap:
[[0.18123744]]
pobj1: [[2.00049253e+15]], pobj2: [[0.25714037]], dobj: [[0.15162025]], gap:
[[0.10552012]]
pobj1: [[4.09182234e+15]], pobj2: [[0.24638435]], dobj: [[0.15923124]], gap:
[[0.08715311]]
pobj1: [[6.66879518e+16]], pobj2: [[0.24425146]], dobj: [[0.16203391]], gap:
[[0.08221754]]
pobj1: [[2.46328846e+16]], pobj2: [[0.2395561]], dobj: [[0.16530816]], gap:
[[0.07424795]]
pobj1: [[5.56415669e+15]], pobj2: [[0.22974238]], dobj: [[0.1676167]], gap:
[[0.06212568]]
pobj1: [[8.52730573e+14]], pobj2: [[0.22288229]], dobj: [[0.16920733]], gap:
[[0.05367496]]
pobj1: [[1.3627501e+17]], pobj2: [[0.21853452]], dobj: [[0.17051376]], gap:
[[0.04802076]]
pobj1: [[1.65569808e+15]], pobj2: [[0.21599477]], dobj: [[0.1708118]], gap:
[[0.04518297]]
pobj1: [[1.19251262e+17]], pobj2: [[0.211691]], dobj: [[0.17153159]], gap:
[[0.04015942]]
pobj1: [[1.41833894e+15]], pobj2: [[0.20720215]], dobj: [[0.17208954]], gap:
[[0.03511261]]
pobj1: [[3.46751557e+13]], pobj2: [[0.19855861]], dobj: [[0.17468729]], gap:
[[0.02387132]]
pobj1: [[9.03382983e+11]], pobj2: [[0.19265609]], dobj: [[0.18151853]], gap:
[[0.01113756]]
solved
pobj1: [[2.36523613e+12]], pobj2: [[0.19165336]], dobj: [[0.18349968]], gap:
[[0.00815368]]
```

[26]:
```
fig,ax=plt.subplots(figsize=(12,10))
plt.plot(volume_index,price_series,label='Original')
plt.plot(volume_index,adaptive_trend_filter_series,label='Adaptive Penalzied␣
↪off Prior',lw=5)
plt.title("Adaptive Penalized Estimation")
plt.xlabel("Time")
plt.ylabel('Price')
plt.legend()
```

[26]: <matplotlib.legend.Legend at 0x7febbad48f40>



[ ]:
```

```

0.0.10 Step 7; Changepoint Extraction

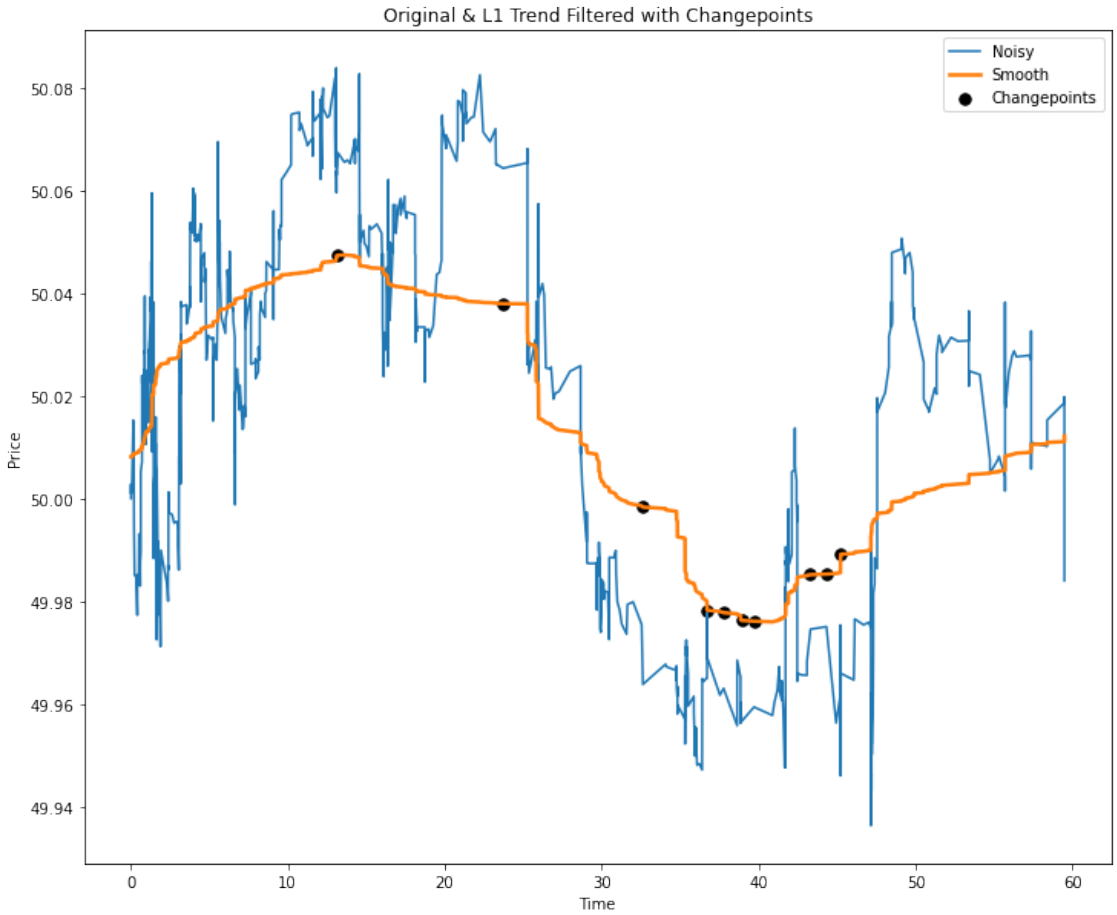
- As estimator is piecweise linear; changepoints can be recovered in both simulations with second order difference operator
- Recovered changepoints plotted below; marked in black
- Threshold for occurrence of changepoint is another hyperparameter to be tuned

[27]:
```
def extract_cp(smooth,difference_order=2,threshold=1e-6):
    """ Extract changepoints via difference operator
    """
    diff_mat=Dmat(len(smooth),2).todense()
    diff=np.dot(diff_mat,smooth).reshape(1,-1)[0]

    x,y,index=np.where([abs(diff)>threshold])
    return index
```

[28]:
```
changepoints=extract_cp(trend_filter_series)
adaptive_changepoints=extract_cp(adaptive_trend_filter_series,threshold=1e-4)
```

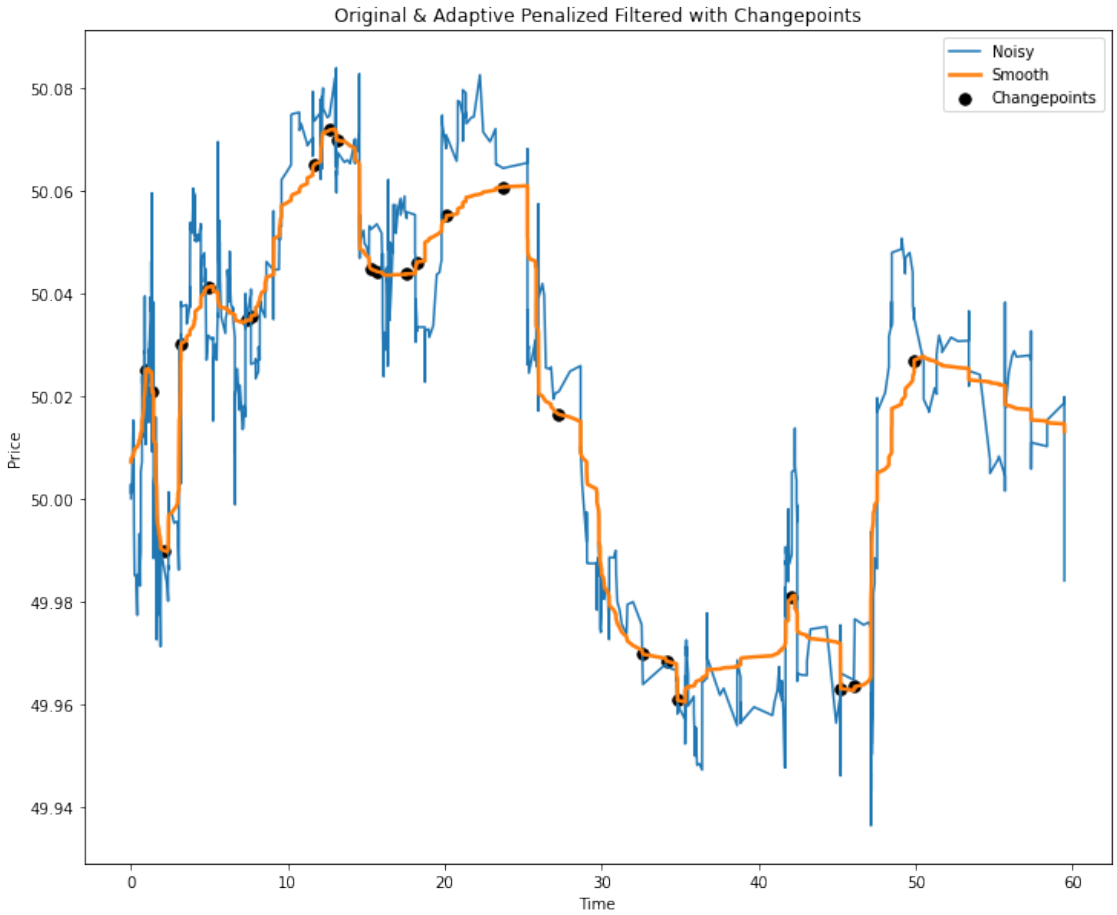
[29]:
```
fig,ax=plt.subplots(figsize=(12,10))
plt.plot(volume_index,price_series,label='Noisy')
plt.plot(volume_index,trend_filter_series,label='Smooth',lw=2.5)
plt.scatter(np.
 ↪array(volume_index)[changepoints],trend_filter_series[changepoints],color='black',s=55,labe
plt.title("Original & L1 Trend Filtered with Changepoints")
plt.xlabel("Time")
plt.ylabel('Price')
plt.legend()
```

[29]:
```
<matplotlib.legend.Legend at 0x7febbad02170>
```



[30]:
```
fig,ax=plt.subplots(figsize=(12,10))
plt.plot(volume_index,price_series,label='Noisy')
plt.plot(volume_index,adaptive_trend_filter_series,label='Smooth',lw=2.5)
plt.scatter(np.
 ↪array(volume_index)[adaptive_changepoints],adaptive_trend_filter_series[adaptive_changepoin
plt.title("Original & Adaptive Penalized Filtered with Changepoints")
plt.xlabel("Time")
plt.ylabel('Price')
plt.legend()
```

[30]: <matplotlib.legend.Legend at 0x7febca2c7a60>


Original & Adaptive Penalized Filtered with Changepoints
Noisy
Smooth
Changepoints
Price
Time
50.08
50.06
50.04
50.02
50.00
49.98
49.96
49.94
0
10
20
30
40
50
60

```
[ ]:
```